Java Backend Architecture

Interview Series

Chapter 12.7
RTL Language Support

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

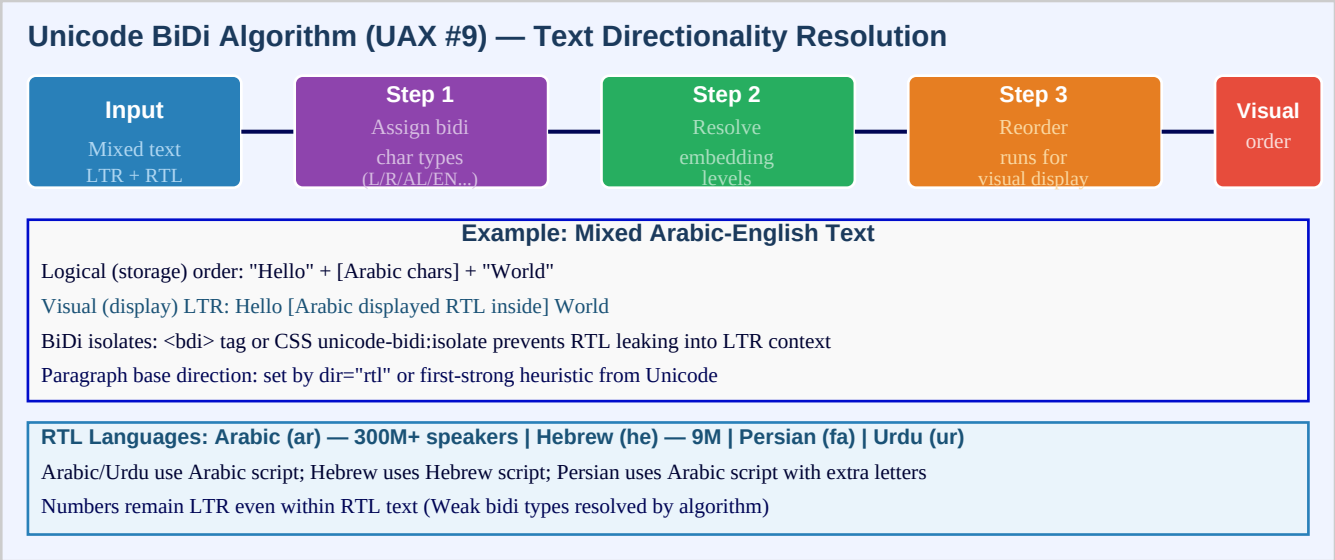
Chapter 12.7 – RTL Language Support

Introduction

Right-to-left (RTL) language support is one of the most frequently underestimated aspects of internationalisation. Arabic, with over 300 million native speakers, Hebrew, Persian, and Urdu all write from right to left. When a web application is used in these locales, the entire layout must mirror: navigation bars move to the right, sidebars appear on the left (logical start in RTL becomes the right edge), padding and margin values swap sides, and directional icons (back/forward arrows, scroll indicators) must be flipped. The Unicode Bidirectional Algorithm (UAX #9) handles mixed-direction text automatically — Arabic words inside an otherwise English sentence are displayed right-to-left — but the containing block direction must be explicitly set by the application.

Modern CSS provides a clean path to RTL support through logical properties: margin-inline-start instead of margin-left, padding-inline-end instead of padding-right, and text-align: start instead of text-align: left. These properties automatically adapt to the document direction. Spring MVC detects the user's locale from the Accept-Language header (or from a cookie, session, or query parameter via LocaleChangeInterceptor), stores it in LocaleContextHolder (a ThreadLocal), and the view layer can read the locale to set the dir attribute on the HTML root element. Testing RTL requires more than translation — visual regression tests that capture screenshots for Arabic and Hebrew locales are essential to catch layout regressions.

Unicode BiDi Algorithm



CSS Physical vs Logical Properties for RTL

CSS RTL Support: Physical vs Logical Properties

Physical Properties (Avoid)

```
margin-left: 16px     (breaks in RTL)
padding-right: 8px    (wrong side)
text-align: left      (use start)
border-left: ...       (use inline-start)
float: left            (use inline-start)
```

Logical Properties (Prefer)

```
margin-inline-start: 16px   (LTR:left, RTL:right)
padding-inline-end: 8px     (LTR:right, RTL:left)
text-align: start           (follows dir)
border-inline-start: ...    (start side)
float: inline-start         (logical)
```

HTML dir Attribute & CSS direction

<html dir="rtl"> or <html lang="ar"> — sets base direction for entire page
 — scoped RTL section within LTR page
CSS: html[dir="rtl"] .nav { flex-direction: row-reverse; } — RTL-specific override

Icon Mirroring: Directional icons must be flipped for RTL — back arrow, forward, next/prev, scroll, search field icon
Non-directional icons (save, star, settings gear) must NOT be mirrored; clocks, checkmarks are non-directional

Spring MVC Locale Detection & RTL Pipeline

Spring MVC: Locale Detection & RTL Handling Pipeline

RTL Testing Strategy

1. Unit: MockMvc with Accept-Language: ar; assert dir="rtl" in HTML
2. Visual: screenshot diff for Arabic/Hebrew layout vs English baseline
3. BiDi: test mixed LTR/RTL strings (user name embedded in RTL sentence)
4. Numbers: Arabic-Indic numerals vs Western (locale-specific number format)
5. Icon: verify back arrow flipped; checkmark not flipped

LTR isolation: <bdi> element or unicode-bidi:isolate prevents RTL username from corrupting surrounding LTR sentence

Key Definitions

RTL	Right-to-Left text direction used in Arabic (ar), Hebrew (he), Persian (fa), and Urdu (ur); text begins at the right margin
BiDi	Bidirectional text — text that contains both LTR and RTL segments; handled by the Unicode Bidirectional Algorithm (UAX #9)
UAX #9	Unicode Standard Annex #9 — the specification defining the Bidirectional Algorithm; determines display order of mixed-direction text
dir attribute	HTML attribute: dir="ltr", dir="rtl", or dir="auto"; sets the base text direction for the element and its descendants
direction (CSS)	CSS property: direction: rtl | ltr; sets the inline-axis direction; should be paired with the HTML dir attribute
unicode-bidi	CSS property controlling how inline text direction is handled; values: normal, embed, bidi-override, isolate, isolate-override

Logical properties	CSS properties relative to writing direction: margin-inline-start, padding-block-end, border-inline; automatically flip in RTL
text-align: start	Aligns text to the start of the line box — left in LTR, right in RTL; preferred over text-align: left in i18n CSS
element	HTML element (Bidirectional Isolate): isolates its content from the surrounding text direction; prevents RTL user names from corrupting LTR context
Icon mirroring	Directional icons (back arrow, forward, scroll, search icon in field) must be horizontally flipped for RTL; decorative/abstract icons should not be
LocaleContextHolder	Spring class that stores the current Locale in a ThreadLocal; accessible from any Spring component via LocaleContextHolder.getLocale()
LocaleChangeInterceptor	Spring MVC interceptor that reads a "lang" query parameter (or custom param) and updates the locale resolver with the new locale
AcceptHeaderLocaleResolver	Spring locale resolver that reads the Accept-Language HTTP header; the simplest and stateless resolver
CookieLocaleResolver	Spring locale resolver that stores the user's locale in a browser cookie; persists across requests without authentication
LTR isolation	Technique to prevent RTL content (like an Arabic username) from changing the direction of surrounding LTR text; use or unicode-bidi:isolate

Locale Resolver Comparison

Resolver	Source	Persistence	Override?	Best For
AcceptHeaderLocaleResolver	Accept-Language header	None (stateless)	No	APIs, no user pref storage
CookieLocaleResolver	Browser cookie	Browser session	Yes	Web apps, anonymous users
SessionLocaleResolver	HTTP session attribute	Session lifetime	Yes	Web apps, logged-in users
FixedLocaleResolver	Config (hardcoded)	Permanent	No	Single-locale apps, tests
DB-backed (custom)	User profile DB column	Permanent	Yes	Multi-tenant, authenticated

Code Examples

1. Spring MVC locale configuration with LocaleChangeInterceptor

```
@Configuration @EnableWebMvc public class WebConfig implements WebMvcConfigurer { // CookieLocaleResolver: persists user locale choice in a cookie @Bean public LocaleResolver localeResolver() { CookieLocaleResolver resolver = new CookieLocaleResolver("lang"); resolver.setDefaultLocale(Locale.ENGLISH); resolver.setCookieMaxAge(Duration.ofDays(365)); return resolver; } // LocaleChangeInterceptor: ?lang=ar switches locale @Bean public LocaleChangeInterceptor localeChangeInterceptor() { LocaleChangeInterceptor interceptor = new LocaleChangeInterceptor(); interceptor.setParamName("lang"); // GET /page?lang=ar return interceptor; } @Override public void addInterceptors(InterceptorRegistry registry) { registry.addInterceptor(localeChangeInterceptor()); } } // Usage: http://myapp.com/dashboard?lang=ar // → sets cookie "lang=ar"; subsequent requests use Arabic locale // Reading locale in a controller @GetMapping("/content") public String
```

```
getContent(HttpServletRequest req, Model model) { Locale locale =
LocaleContextHolder.getLocale(); // from ThreadLocal boolean isRtl = isRtlLocale(locale);
model.addAttribute("dir", isRtl ? "rtl" : "ltr"); model.addAttribute("lang",
locale.getLanguage()); return "content"; } private boolean isRtlLocale(Locale locale) { String
lang = locale.getLanguage(); return Set.of("ar", "he", "fa", "ur").contains(lang); }
```

2. HTML templates with dir attribute and BDI isolation

```
<!-- Thymeleaf template: dir set from server-side locale detection --> <!DOCTYPE html> <html
th:lang="${lang}" th:dir="${dir}"> <!-- th:dir will be "rtl" for Arabic/Hebrew or "ltr" for
others --> <head> <meta charset="UTF-8"> <!-- CSS: use logical properties for automatic RTL
support --> <style> .card { margin-inline-start: 16px; /* left in LTR, right in RTL */
padding-inline-end: 12px; /* right in LTR, left in RTL */ border-inline-start: 3px solid
#2980B9; text-align: start; /* left in LTR, right in RTL */ } /* RTL-specific icon flip */
[dir="rtl"] .icon-back { transform: scaleX(-1); } [dir="rtl"] .icon-next { transform:
scaleX(-1); } /* Non-directional icons: DO NOT flip */ /* .icon-save, .icon-star,
.icon-settings — no flip needed */ </style> </head> <body> <!-- BDI: isolate user-submitted RTL
text within LTR sentence --> <p>Posted by <bdi th:text="${username}">username</bdi> at
10:30am</p> <!-- Without <bdi>: if username is Arabic, "10:30am" shifts to left edge --> <!--
With <bdi>: Arabic name displayed RTL, rest of sentence stays LTR --> <!-- Scoped RTL section
within LTR page --> <div dir="rtl" lang="ar"> <p>[redacted]</p> <!-- Arabic text within
RTL div --> <p>Hello <bdi>World</bdi></p> <!-- LTR word isolated within RTL context --> </div>
</body> </html>
```

3. CSS logical properties and RTL layout

```
/* ============ BEFORE: Physical properties (breaks in RTL) ============ */ .nav-item {
margin-left: 16px; /* WRONG: always left regardless of direction */ padding-right: 8px; /*
WRONG: always right */ border-left: 2px solid; /* WRONG: always left border */ text-align:
left; /* WRONG: doesn't follow direction */ } /* ============ AFTER: Logical properties
(direction-aware) ============ */ .nav-item { margin-inline-start: 16px; /* left in LTR →
right in RTL */ padding-inline-end: 8px; /* right in LTR → left in RTL */ border-inline-start:
2px solid; /* left in LTR → right in RTL */ text-align: start; /* left in LTR → right in RTL
*/ } /* Block-axis logical properties */ .section { margin-block-start: 16px; /* replaces
margin-top */ margin-block-end: 8px; /* replaces margin-bottom */ padding-block: 12px; /*
shorthand for block-start and block-end */ } /* Flexbox RTL: use logical values */ .toolbar {
display: flex; flex-direction: row; /* in RTL, items flow right-to-left automatically */ gap:
8px; justify-content: flex-start; /* start of flex container (right in RTL) */ } /* When you
must target RTL specifically */ [dir="rtl"] .brand-logo { margin-inline-start: 0;
margin-inline-end: auto; } /* unicode-bidi for inline RTL content */ .rtl-text { direction:
rtl; unicode-bidi: embed; /* create a new embedding level */ } /* Isolate user content from
surrounding direction */ .user-comment { unicode-bidi: isolate; /* equivalent to <bdi>
behaviour */ }
```

4. Number formatting in RTL contexts

```
import java.text.NumberFormat; import java.util.Locale; // RTL locales with different number
formatting conventions Locale arabic = new Locale("ar", "SA"); // Arabic (Saudi Arabia) Locale
hebrew = new Locale("he", "IL"); // Hebrew (Israel) Locale persian = new Locale("fa", "IR"); //
Persian (Iran) Locale english = Locale.US; double amount = 1234567.89; // Arabic locale: may
use Arabic-Indic numerals ([redacted]) // Depends on the locale variant and OS NumberFormat
arFmt = NumberFormat.getCurrencyInstance(arabic); NumberFormat heFmt =
NumberFormat.getCurrencyInstance(hebrew); NumberFormat faFmt =
NumberFormat.getNumberInstance(persian); NumberFormat enFmt =
NumberFormat.getCurrencyInstance(english); System.out.println(arFmt.format(amount)); //
```

```
■■■■■■■■■■■■ ■.■.■ (Arabic-Indic) System.out.println(heFmt.format(amount)); //
■1,234,567.89 ■ (Western numerals) System.out.println(faFmt.format(amount)); //
■■■■■■■■■■■■ (Persian-Indic) // In HTML/JS: Intl.NumberFormat handles this automatically //
new Intl.NumberFormat('ar-SA', {style:'currency', currency:'SAR'}).format(1234.56) // Java:
force Western (ASCII) numerals for Arabic locale NumberFormat forceWestern =
NumberFormat.getNumberInstance(arabic); forceWestern.setGroupingUsed(true); // Note: Java's
NumberFormat may produce Arabic-Indic digits depending on JDK version // To force Western: use
ICU4J NumberFormat with symbols override // Spring MessageFormat in RTL: numbers in messages
maintain their directionality // ICU MessageFormat handles RTL number placeholders correctly:
// "{count, plural, one{# ■■■■} other{# ■■■■■}}" — # is the number, direction resolved by
BiDi // Testing: assert formatted numbers are correct for both locales // @ParameterizedTest //
@ValueSource(strings = {"ar-SA", "he-IL", "fa-IR"}) // void testCurrencyFormat(String
localeTag) { // Locale locale = Locale.forLanguageTag(localeTag); // String formatted =
formatCurrency(amount, locale); // assertThat(formatted).isNotNull().isNotEmpty(); // }
```

5. Spring Accept-Language locale resolver and RTL detection

```
import org.springframework.web.servlet.i18n.AcceptHeaderLocaleResolver; @Configuration public
class I18nConfig { @Bean public LocaleResolver localeResolver() { AcceptHeaderLocaleResolver
resolver = new AcceptHeaderLocaleResolver(); resolver.setDefaultLocale(Locale.ENGLISH);
resolver.setSupportedLocales(List.of( Locale.ENGLISH, new Locale("ar"), // Arabic new
Locale("he"), // Hebrew new Locale("fa"), // Persian new Locale("ur"), // Urdu Locale.FRENCH,
Locale.GERMAN, new Locale("zh", "CN"), new Locale("ja"), new Locale("ko") )); return resolver;
} } // Service: determine RTL and set on response model @Component public class
DirectionService { private static final Set<String> RTL_LANGUAGES = Set.of("ar", "he", "fa",
"ur", "yi", "dv", "ha", "khw", "ks", "ku", "ps", "sd", "ug"); public TextDirection
getDirection(Locale locale) { return RTL_LANGUAGES.contains(locale.getLanguage()) ?
TextDirection.RTL : TextDirection.LTR; } } public enum TextDirection { LTR, RTL; public String
getHtmlDir() { return name().toLowerCase(); } } // REST API: include direction in API response
metadata @GetMapping("/api/meta") public ApiMeta getMeta() { Locale locale =
LocaleContextHolder.getLocale(); return ApiMeta.builder() .locale(locale.toLanguageTag()) //
"ar-SA" .direction(directionService.getDirection(locale).getHtmlDir()) // "rtl" .build(); }
```

6. MockMvc RTL testing and visual regression strategy

```
@SpringBootTest @AutoConfigureMockMvc class RtlSupportTest { @Autowired MockMvc mockMvc;
@Autowired MessageSource messages; // Test 1: Accept-Language: ar returns dir=rtl in HTML @Test
void arabicLocaleSetsDirRtl() throws Exception { mockMvc.perform(get("/dashboard")
.header("Accept-Language", "ar-SA")) .andExpect(status().isOk())
.andExpect(xpath("//html/@dir").string("rtl")) .andExpect(xpath("//html/@lang").string("ar"));
} // Test 2: English returns dir=ltr @Test void englishLocaleSetsDirLtr() throws Exception {
mockMvc.perform(get("/dashboard") .header("Accept-Language", "en-US"))
.andExpect(xpath("//html/@dir").string("ltr")); } // Test 3: Parameterized across RTL locales
@ParameterizedTest @ValueSource(strings = {"ar", "he", "fa", "ur"}) void
rtlLanguagesProduceDirRtl(String lang) throws Exception { mockMvc.perform(get("/dashboard")
.header("Accept-Language", lang)) .andExpect(xpath("//html/@dir").string("rtl")); } // Test 4:
Messages available for Arabic locale @Test void arabicMessagesLoaded() { Locale arabic = new
Locale("ar"); String greeting = messages.getMessage("greeting.hello", null, arabic);
assertThat(greeting).isNotNull().isNotEmpty(); // Should be Arabic text, not the key
"greeting.hello" assertThat(greeting).doesNotContain("greeting.hello"); } // Test 5: API
metadata includes direction @Test void apiMetaIncludesRtlDirection() throws Exception {
mockMvc.perform(get("/api/meta") .header("Accept-Language", "he")
.accept(MediaType.APPLICATION_JSON)) .andExpect(jsonPath("$.direction").value("rtl"))
.andExpect(jsonPath("$.locale").value("he")); } }
```

Interview Questions & Answers

Q1. Which languages require RTL support and what distinguishes their scripts?

The primary RTL languages are Arabic (ar) — written in the Arabic script, over 300 million native speakers; Hebrew (he) — written in the Hebrew script, about 9 million speakers; Persian/Farsi (fa) — uses the Arabic script with additional letters; Urdu (ur) — uses the Nastaliq variant of the Arabic script. Other RTL languages include Yiddish (yi), Pashto (ps), Sindhi (sd), and Uyghur (ug). Key differences: Arabic and Persian connect letters (cursive); Hebrew is a non-cursive alphabet. Numbers in all these scripts are written left-to-right even within RTL text — the BiDi algorithm handles this automatically as 'weak' directional characters.

Q2. What is the Unicode Bidirectional Algorithm and why is it needed?

The Unicode Bidirectional Algorithm (Unicode Annex #9, UAX #9) defines how a sequence of characters with mixed directionality is displayed. In a sentence like 'Order shipped to Ahmed (■■■■) on Monday', the English words are LTR, the Arabic name is RTL, but the sentence structure is LTR. The algorithm assigns a bidi category to each character (L = left-to-right, R = right-to-left, AL = Arabic letter, EN = European number, etc.), resolves embedding levels, and reorders the characters for visual display while keeping the logical (storage) order intact. This is handled by the browser/OS rendering engine automatically — developers must set the base direction (dir attribute) correctly for the algorithm to work.

Q3. What is the difference between dir='rtl' on HTML vs a CSS direction:rtl rule?

HTML dir='rtl' is a semantic attribute that affects: (1) the base direction for the BiDi algorithm — determining how mixed text is laid out; (2) the alignment of block elements; (3) the layout of flex/grid containers; (4) the UA stylesheet — inputs and form elements get RTL layout. CSS direction: rtl affects the CSS layout engine but not the HTML BiDi algorithm directly. Best practice: set the dir attribute on the element and mirror it with CSS :lang() or [dir=rtl] selectors for CSS-only overrides. Use both together: in markup and html { direction: rtl; } in CSS. Never rely on CSS direction alone without the HTML attribute.

Q4. What are CSS logical properties and why should they be used for RTL support?

CSS logical properties map layout dimensions to the writing mode instead of physical directions. Physical: margin-left, padding-right, border-top. Logical: margin-inline-start (left in LTR, right in RTL), padding-inline-end (right in LTR, left in RTL), border-block-start (equivalent to border-top in horizontal writing). Benefits: write CSS once; it works for both LTR and RTL without separate [dir=rtl] overrides. Block axis (block-start/end) replaces top/bottom in vertical writing modes (CJK). Inline axis (inline-start/end) replaces left/right. Support: all modern browsers. Shorthand: margin-inline: 16px; padding-block: 8px. Migration: replace margin-left with margin-inline-start, text-align:left with text-align:start.

Q5. What is the HTML element and when should you use it?

The Bidirectional Isolate element () isolates a span of text from the surrounding text direction. This is critical when embedding user-generated content (names, comments) into a fixed-direction sentence. Example: 'Posted by Ahmed (■■■■)' — if '■■■■' is an Arabic username without , the parentheses shift position and the sentence renders incorrectly. With Posted by ■■■■ at 10am, the Arabic name is isolated. The CSS equivalent is unicode-bidi: isolate. Use whenever rendering user-supplied names, comments, or any content whose directionality is unknown at template-authoring time.

Q6. How does Spring MVC detect the user's locale from the Accept-Language header?

AcceptHeaderLocaleResolver reads the Accept-Language HTTP header (e.g., 'ar-SA,ar;q=0.9,en;q=0.8') and selects the best matching locale from the configured supported locales using quality values (q values). It calls Locale.lookup() (RFC 4647 language range lookup). Configuration: resolver.setSupportedLocales(List.of(new Locale('ar'), Locale.ENGLISH, ...)) and resolver.setDefaultLocale(Locale.ENGLISH). The resolved locale is

stored in LocaleContextHolder via LocaleContextHolder.setLocale(). Limitation: AcceptHeaderLocaleResolver is stateless — it does not remember the user's preference between requests without cookies or session. For user preference persistence, use CookieLocaleResolver or SessionLocaleResolver.

Q7. What is LocaleContextHolder and how does it work in a multithreaded server?

LocaleContextHolder is a Spring utility that stores the current locale in a ThreadLocal. Since each HTTP request is processed on its own thread (in a servlet container), the ThreadLocal is request-scoped: set at the start of the request by the DispatcherServlet (using the LocaleResolver), readable throughout the request processing chain via LocaleContextHolder.getLocale(), and cleared at the end of the request. In reactive (WebFlux) applications, ThreadLocal does not work because a single request can be processed across multiple threads. Use Reactor Context instead: Mono.deferContextual() to read the locale from the reactive context propagated from the entry point.

Q8. Which icons should be mirrored for RTL and which should not be?

Mirror directional icons that represent movement or navigation: back arrow, forward arrow, next/previous page buttons, audio/video forward/rewind, scroll arrows, breadcrumb separators, text indent/outdent icons, search bar with icon on left (moves to right), list item bullet pointing right (should point left in RTL). Do NOT mirror: checkmarks, stars, settings gear, save floppy disk, play button (triangle is non-directional), calendar, clock, camera, microphone, phone, lock. Rule: if the icon represents something with an inherent directionality in the real world (pointing, moving) — mirror it. If it is a symbolic or abstract representation — do not mirror. Google Material Design and Apple Human Interface Guidelines both document mirroring rules for their icon sets.

Q9. How do you handle number formatting in RTL languages?

Number formatting in RTL locales is handled by java.text.NumberFormat or java.text.DecimalFormat with the appropriate locale. Arabic (ar-SA) may use Arabic-Indic digits (■■■■■■■■■■) while Arabic (ar-EG) uses Western digits depending on locale variant. Persian (fa-IR) uses Extended Arabic-Indic digits (■■■■■■■■). Hebrew uses Western digits. Numbers within RTL text are rendered LTR by the BiDi algorithm (they are 'weak' characters). For currency: NumberFormat.getCurrencyInstance(new Locale('ar', 'SA')) applies the correct currency symbol and placement. For Thymeleaf: #numbers.formatDecimal(value, 1, 2) uses the current locale automatically.

Q10. How do you test that an application correctly supports RTL layout?

Testing strategy has multiple layers: (1) Unit: assert that RTL locales produce dir='rtl' in HTML output using MockMvc + XPath assertions. (2) Integration: verify message source has translations for all keys in Arabic/Hebrew. (3) Visual regression: take screenshots of key pages in Arabic and Hebrew locales; compare with approved baselines using tools like Percy, Chromatic, or Playwright screenshots. (4) BiDi text tests: verify that usernames containing Arabic characters are properly isolated with in user-facing pages. (5) Icon tests: manually verify mirrored icons using the RTL locale in the browser. (6) Form tests: verify input fields align to the right and label positions are correct. (7) Number/currency: assert formatted amounts are correct for ar-SA locale.

Q11. What is the difference between AcceptHeaderLocaleResolver, CookieLocaleResolver, and SessionLocaleResolver?

AcceptHeaderLocaleResolver: reads Accept-Language header; stateless (no persistence); cannot be changed programmatically during a session; best for pure APIs where the client manages locale. CookieLocaleResolver: stores the locale in a browser cookie; persists across browser sessions; user can change locale via ?lang=ar and it sticks; does not require authentication. SessionLocaleResolver: stores locale in HTTP session attribute; cleared when session expires; requires the user to be in a session. Custom/DB-backed: stores locale preference in users table; retrieved after authentication; most reliable for authenticated users. Best practice: use AcceptHeaderLocaleResolver for APIs; CookieLocaleResolver for

public web apps; DB-backed for authenticated multi-tenant apps.

Q12. How should a REST API signal RTL to a frontend JavaScript application?

The API should include direction metadata in responses so the frontend can set the HTML dir attribute. Approaches: (1) Add a 'dir' field to a locale/meta endpoint: GET /api/locale returns {locale: 'ar-SA', direction: 'rtl', dateFormat: 'dd/MM/yyyy'}. (2) Include in JWT claims: {locale: 'ar', dir: 'rtl'} so the frontend has it immediately on login. (3) Include in OpenAPI spec so frontend knows which locales are RTL. (4) The frontend JavaScript can also detect RTL from the locale tag itself using: const RTL_LANGS = ['ar','he','fa','ur']; const isRtl = RTL_LANGS.includes(locale.split('-')[0]); document.documentElement.dir = isRtl ? 'rtl' : 'ltr';

Q13. How do you handle RTL in locale-aware date and time display?

Date formats for RTL locales: Arabic (ar-SA) uses dd/MM/yyyy (Gregorian) or the Hijri (Islamic) calendar; Hebrew (he-IL) uses dd/MM/yyyy; Persian uses the Solar Hijri calendar. Java: DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT).withLocale(locale) automatically applies the locale's format. For Islamic calendar: use java.time.chrono.HijrahChronology or ThaiBuddhistChronology for Thai locale. In RTL contexts, date components may render right-to-left but the day/month/year order follows locale convention. For UI: Intl.DateTimeFormat in JavaScript handles both the calendar system and the number system for the locale.

Q14. What is the unicode-bidi CSS property and its values?

The unicode-bidi property controls how the Unicode Bidirectional Algorithm applies to an element's content. Values: normal — element doesn't open a new embedding level (default); embed — opens a new embedding level; the direction is specified by the direction property; bidi-override — overrides the implicit BiDi reordering; characters display in the direction property order; isolate — element is treated as an isolated unit from the perspective of the surrounding text (equivalent to); isolate-override — combination of isolate and bidi-override; plaintext — base direction determined by first-strong heuristic from content. Most common: unicode-bidi: isolate for wrapping user content; unicode-bidi: embed with direction: rtl for a block of RTL content.

Q15. How do you configure message bundles for Arabic and Hebrew in Spring Boot?

1. Create message files: messages_ar.properties (Arabic), messages_he.properties (Hebrew), messages_fa.properties (Persian) alongside messages.properties (default). 2. Set encoding: spring.messages.encoding=UTF-8 in application.properties. 3. Configure MessageSource: spring.messages.basename=messages; spring.messages.fallback-to-system-locale=false. 4. For right-to-left message content, no special configuration needed — the text in the .properties file is just Unicode. However: pluralization rules differ across languages. Arabic has 6 plural forms; Hebrew has 4. Use ICU MessageFormat for complex plural handling: '{count, plural, zero{■■ ■■■■■} one{■■■■■ ■■■■■} two{■■■■■■■■} few{# ■■■■■} many{# ■■■■■} other{# ■■■■■}}'.

Q16. What are common mistakes made when implementing RTL support?

(1) Using physical CSS properties (margin-left, padding-right) instead of logical ones — breaks RTL layout without [dir=rtl] overrides. (2) Setting dir='rtl' in CSS only without the HTML attribute — breaks BiDi algorithm. (3) Forgetting around user-generated content — Arabic usernames corrupt surrounding LTR text. (4) Mirroring non-directional icons (checkmarks, settings) — looks wrong in RTL. (5) Not mirroring directional icons (back arrow) — confusing in RTL. (6) Hardcoding text alignment (text-align: left) — should be text-align: start. (7) Assuming all RTL locales use the same numeric system — Arabic-Indic vs Western. (8) No visual regression tests — RTL layout bugs are invisible in unit tests alone. (9) Not testing BiDi with mixed content — pure Arabic pages may work but mixed fail.

Q17. How does LocaleChangeInterceptor work and how do you secure it?

LocaleChangeInterceptor is a Spring HandlerInterceptor that checks for a locale parameter (?lang=ar) in every request and calls the LocaleResolver.setLocale() method if the parameter is present. Configuration: interceptor.setParamName('lang'); register via addInterceptors(). Security considerations: (1) Validate the locale parameter against a whitelist of supported locales — an attacker could send any string as ?lang=xyz. Spring's resolver.setSupportedLocales() handles this; unsupported locales fall back to default. (2) For authenticated users, also save the locale preference to the database. (3) Rate limiting on locale changes prevents abuse. (4) The interceptor should be listed before security-sensitive interceptors in the chain.

Q18. How do you propagate locale in async and reactive Spring contexts?

ThreadLocal-based LocaleContextHolder does not propagate across thread boundaries in async (@Async) or reactive (WebFlux) code. Solutions: For @Async: configure a TaskDecorator on the thread pool executor that copies the LocaleContext from the parent thread to the child thread: executor.setTaskDecorator(r -> { LocaleContext lc = LocaleContextHolder.getLocaleContext(); return () -> { LocaleContextHolder.setLocaleContext(lc); r.run(); } }); For WebFlux / Project Reactor: store the locale in Reactor Context: Mono.deferContextual(ctx -> { Locale locale = ctx.get('locale'); ... }). Propagate from entry filter: Mono.from(...).contextWrite(Context.of('locale', resolvedLocale)). Spring Security's ReactiveSecurityContextHolder follows the same pattern.

Q19. What is the 'first-strong heuristic' in the BiDi algorithm?

When no explicit base direction is set (no dir attribute), the Unicode Bidirectional Algorithm uses the 'first-strong heuristic': it scans the text from the beginning and uses the first character with a strong directionality (L = LTR or R/AL = RTL) as the paragraph base direction. This works well for purely Arabic or purely English text. It fails for: (1) Text that starts with numbers (neutral direction) — the first strong character may be deeper in the text. (2) Mixed content where a product name starts with an English trademark even in an Arabic page. Fix: always set dir explicitly — — rather than relying on heuristics. HTML5 dir='auto' applies first-strong heuristic per element, useful for user-submitted content of unknown direction.

Q20. How do you handle font support for Arabic and Hebrew rendering?

Arabic and Hebrew require fonts that support their Unicode blocks. Built-in system fonts on modern OSes generally cover Arabic (Arial, Times New Roman) and Hebrew (Arial, Tahoma). For web applications: use Google Fonts — Noto Sans Arabic, Cairo, Amiri for Arabic; Frank Ruhl Libre, Heebo for Hebrew. Load via @font-face or Google Fonts CDN. For PDF generation (e.g., with Apache PDFBox or iText): embed fonts that support Arabic/Hebrew scripts — DejaVu Sans or Noto Sans include Arabic glyphs. Java's default Helvetica/Times in PDFBox does not include Arabic glyphs. For ReportLab (Python PDF): use TTFont with an Arabic-supporting TTF file. Test by rendering actual Arabic/Hebrew text and verifying no tofu (empty boxes) appear.